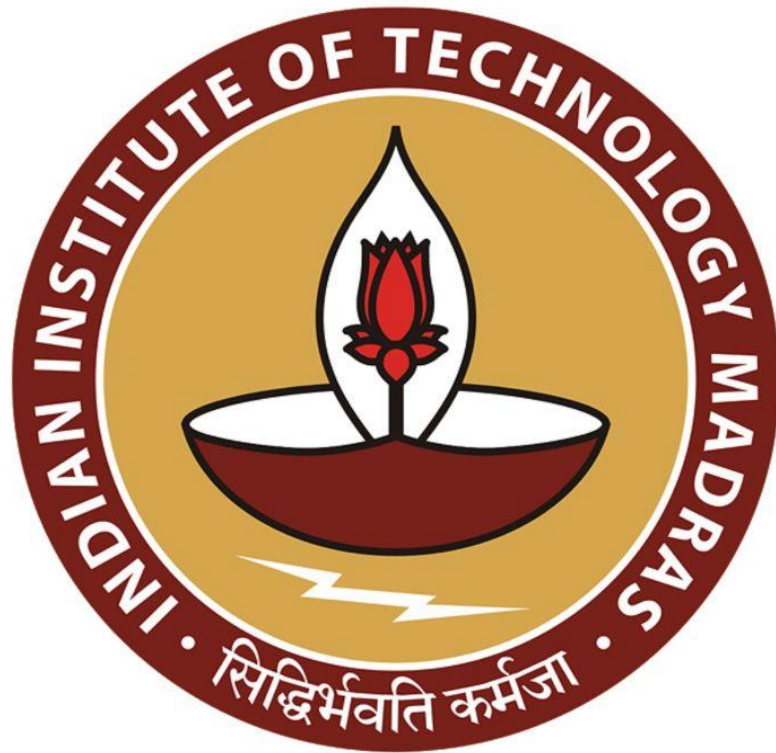




IIT Madras

ONLINE DEGREE

Machine learning techniques
Professor Arun Rajkumar
Department of Computer Science and Engineering
Indian Institute of Technology, Madras
Optimizing the error function

(Refer Slide Time: 00:13)

$$\min_{w \in \mathbb{R}^d} \sum_{i=1}^n (w^T x_i - y_i)^2 = \|Xw - y\|_2^2$$

$$X^T = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix} \quad \begin{matrix} n \times d \\ d \times 1 \end{matrix}$$

$$y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \quad n \times 1$$

So, how do we solve this problem? So, that's the next question. So, we have put down the problem, how do we solve it. So, it is useful to think of this the same objective in terms of matrices, it will make our life a little bit easier to work with matrices as we will see. So, think of let us, let me call a matrix x as just the matrix where the data points are stacked in columns. In other words, x^T is a matrix where the data points are stacked in rows. So, x_1, x_2 dot dot dot x_n , we have n different data points. Now, we also have the label vector y .

So, this is $n \times d$ matrix, we have a label vector y , which gives the labels for each of our data points we can call this vector y . So, this $n \times 1$ vector, now, our parameter is a vector w_1, w_2 dot dot dot w_t , which is a vector w , which is an $d \times 1$. So, basically, what we are saying is that you can rewrite this objective as follows, we can rewrite this as $x^T w - y$ whose this is a vector whose norm squared we are trying to minimize.

So, why because $x^T w$ would give me $x_1^T w, x_2^T w$ and so on. So, if you just think of this, as I mean, it is easier to think of this as just a vector w and $x^T w$ is $x_1^T w, x_2^T w$ vector containing all data points, dot product w , and then we are comparing it with y , corresponding y_i and then squaring it, which can be thought of as just the L two norm squared of this vector, which is which is an n dimensional vector.

(Refer Slide Time: 02:21)



Whiteboard content:

$$\min_{w \in \mathbb{R}^d} \sum_{i=1}^n (x_i^T w - y_i)^2 = \|Xw - y\|_2^2$$
$$X^T = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{bmatrix} \quad \begin{matrix} n \times d & d \times 1 \end{matrix}$$
$$y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \quad n \times 1$$
$$\min_{w \in \mathbb{R}^d} (Xw - y)^T (Xw - y) \quad \leftarrow \text{unconstrained Quadratic (n.w.) Optimisation problem}$$

IIT Madras B.S. Degree



Whiteboard content:

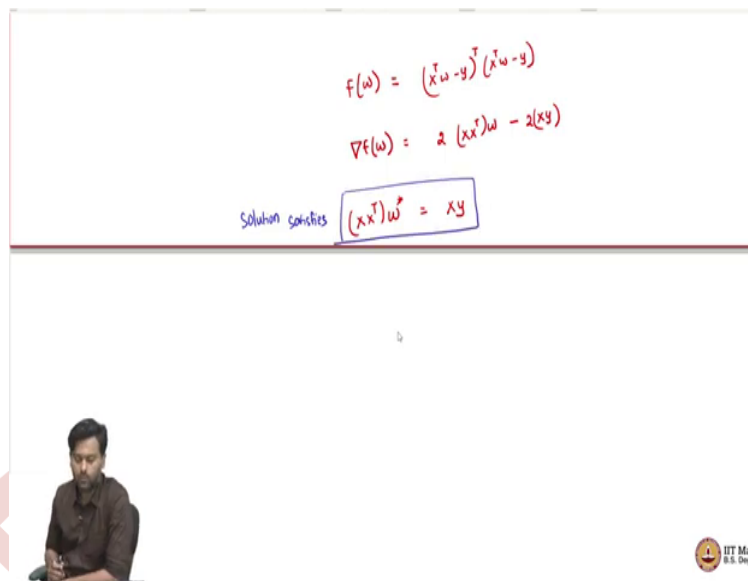
$$\min_{w \in \mathbb{R}^d} (Xw - y)^T (Xw - y) \quad \leftarrow \text{unconstrained Quadratic (n.w.) Optimisation problem}$$

Solution: Take derivative (gradient) and set to zero

$$f(w) = (Xw - y)^T (Xw - y)$$
$$\nabla f(w) = 2(X^T X)w - 2(X^T y)$$

IIT Madras B.S. Degree

सिद्धिर्भवति कर्मजा



Which means basically, what we are seeing is that we want to minimize over w in $\mathbb{R}^d$ this quantity, now, the norm squared can also be written as a vector transpose itself. So, here the vector of interest is $(x^T w - y)^T (x^T w - y)$.

So, now, if you think about this, again, this is an unconstrained quadratic optimization problems why is it unconstrained because we allow for any w and $\mathbb{R}^d$ there is no condition on w that we are imposing. It is quadratic because of the squared error that we have picked.

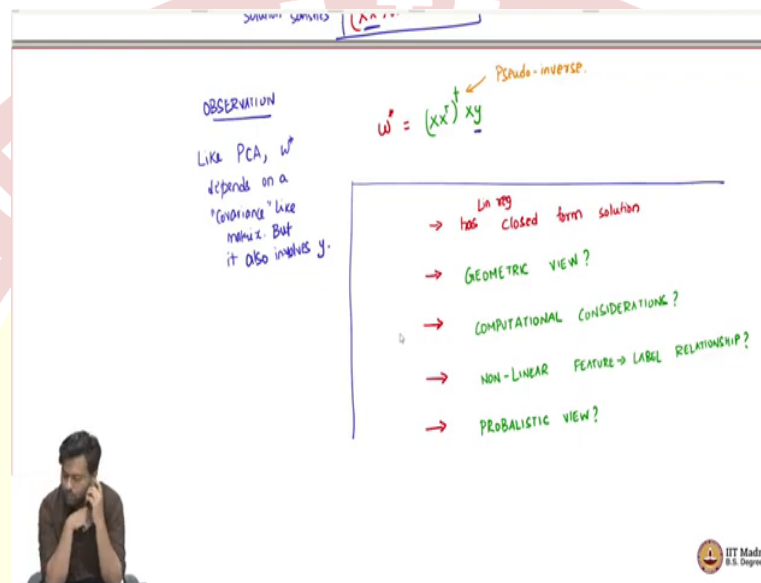
And it is an optimization problem, of course, we want to find the minimum w that has the, that I mean, that minimizes the w that minimizes this objective function. Now, how can we find the w that has the smallest value among this? Well, because it is unconstrained, it is quadratic.

Simplest way is one can start thinking about is take the derivative with respect to w , which derivative in higher dimension is just the gradient and set to 0 and see what happens, so that should give us the optimal w and in some cases, it might be easy, it might give you some closed form solution. In some cases, it might not. But let us see what happens in this case. So, now, let me take the gradient, right. So, now, I am going to think of this as some function of w , which is $(x^T w - y)^T (x^T w - y)$. Now, we will take the gradient.

So, this is the symbol for gradient which is just a vector of partial derivatives with respect to each component of w . You can think of it as I am fixing one component w_i , and then, I mean, I am considering one component w_i and then treating all the other components as constants and trying to take the derivative rate and then collecting all those derivatives in a vector.

If you do that, and this is exercise that you should try, we get something like $2(xx^T)w - 2(xy)$. This would be our gradient. Now we want to set the gradient to 0 if you do that, and then what you would get is you will get the following equation $(xx^T)w = xy$. So, w^* the solution satisfies this equation, so the solution satisfies this equation.

(Refer Slide Time: 05:23)



Now, this means now our w^* can be written as well, because it is a matrix times w^* is x times y , you can try to write w^* in close form. Now, it depends on whether this matrix here is invertible or not. If it is invertible, then you can just pre multiply by the inverse and you can that cancels out one side, if it is not invertible, what is typically used as what is called a pseudo inverse, which is which is denoted like a long cross. You can think of it as inverse for now, we can assume that this is an invertible matrix, nothing conceptually changes so much. We will talk about the pseudo inverse a little later. So, this is the pseudo inverse.

Which, is a matrix which has all good properties that an inverse satisfies when you do not have an inverse, typically, what do you mean you do not have an inverse, it means that there is some null space non trivial null space associated with the matrix or the rank of the matrix is not, it is not full rank and so on.

So, when you cannot invert it, so you can it is like a many to one mapping. So, that is there are so many points which go to the same point, now you have to do an invert, if it is a one to one mapping, the invert inverse would give you the correct value. But then if there are there

is a whole subspace in a finite space, which maps to the same vector on the other side via transformation, then the pseudo inverse would map it to the one that has the least length.

So, that is what the idea of a pseudo inverses, so it is literally the least norm solution is what they call it. Anyway, so this is this is a closed form solution, a lot of things have to be discussed about the solution, which we will do as we move along. The first thing I want you to notice is that so this is a curious observation. So, what is this w^* depending on.

So, this w^* depends on some matrix xx^T . But what is xx^T , x is a matrix of your $d \times n$ matrix x^T is an $n \times d$ matrix, which is like vectors or in rows. So, xx^T is a $d \times d$ matrix, which is like a scale covariance matrix. So, now the interesting observation is that like our PCA, so, the w^* depends on a covariance like matrix. That is the first observation just to connect it to whatever we have seen so far. So, but PCA was completely unsupervised. And this is important to remember.

So, the only difference well, the key difference between PCA and what we are doing now is we also have the labels. So, the labels should also dictate how the answer w^* looks like, because at the end of the day, you are matching the labels as well as possible. Now, the labels also play a role in this which which enters into the picture via y . So, we like PCA, the covariance matrix plays a role or the inverse of the covariance matrix kind of plays a role and we will talk about what is this exact role later on. But now, it suffices to observe that but it also involves the labels y , involves y .

So, which which also enters into the picture. So, given a dataset, I can easily compute w^* , easily within codes, I can compute w^* in closed form solution, So, it might be computationally expensive, and so on, we will talk about that in a minute.

But for now, it looks like at least, I can see this and say that hey, this is my w^* . So, the w^* is a function of the data and the labels $(xx^T)^+ xy$ it looks like close to a mean it is a closed form, but then we we need to make sense of it. So, of course, one observation is that there is some covariance matrix involved and so on. But then that is not like the complete picture definitely. So, we need to understand make sense of this in a better way.

So, how can we make sense of it? So, what what are we saying? So, we are first saying that linear regression has closed form solution, which is a great thing. So, we want to solve a

problem, we have a closed form solution, then we can dig deeper and try to understand what this means.

The first thing we want to understand is, well, this looks complicated $(xx^T)^+ xy$. That needs justification, So, I mean, at least an interpretation. So, we need to first understand what is the geometric view of this solution. So, can we describe the solution geometrically, first question.

The second question is, are there any computational considerations, so it is a closed form solution, that is excellent. But for instance, in PCA, we saw there were some computational constraints. Are there any here. So, what are the computational considerations that we need to think about. The third thing is that well, we have you made a hypothesis that the features are, you know, linearly related to the labels? What if they are not.

So, what if I know that the feature label relationship is quadratic? What if I know it is cubic? So, well, how can we think about nonlinear feature to label relationships? It is a similar type of questions we asked for PCA and answered but then the answers are going to be of different flavor. Naturally should expect that because it is a different problem.

So, but then the question is, are all relevant feature to label relationships? How can we model this and finally, we did talk about overfitting a bit, we did talk about imposing a structure, but then I kind of, did not explain carefully, what did I mean by noise? The moment we want to explain noise better than the right language to understand noise is probability. So, the question is is there a probabilistic way of understanding this problem right. So, is there a probabilistic view?

So, these are four questions we will ask and we will try to answer about linear regression and once we have understood this, then we will have a very solid understanding of linear regression. And that will actually help us build something slightly more sophisticated version of linear regression, which will lead us to a two more algorithmic changes to the standard problem that we have seen here, so in the next the next few videos we are going to think about how to look at each of these questions and see what the answers are.